

# Felhasználói

**Critiqly**

Felhasználói dokumentáció

# Tartalomjegyzék

|                                   |           |
|-----------------------------------|-----------|
| <b>Tartalomjegyzék.....</b>       | <b>2</b>  |
| <b>Kezdőlap / Főoldal.....</b>    | <b>3</b>  |
| Bemutató szöveg:.....             | 3         |
| Kiemelt filmek:.....              | 3         |
| Legjobbra értékelt filmek:.....   | 3         |
| Napi válogatás:.....              | 3         |
| Közelgő filmek:.....              | 3         |
| Navigációt elősegítő gombok:..... | 3         |
| <b>Navigációs sáv.....</b>        | <b>4</b>  |
| Logó:.....                        | 4         |
| Keresősáv:.....                   | 4         |
| Főoldal fül:.....                 | 4         |
| Filme fül:.....                   | 4         |
| Rólunk fül:.....                  | 4         |
| Belépés gomb:.....                | 4         |
| Regisztráció gomb:.....           | 4         |
| <b>Regisztrációs oldal.....</b>   | <b>6</b>  |
| Név:.....                         | 6         |
| Felhasználónév:.....              | 6         |
| E-mail cím:.....                  | 6         |
| Jelszó:.....                      | 6         |
| Jelszó újra:.....                 | 6         |
| <b>Belépés oldal.....</b>         | <b>7</b>  |
| E-mail cím:.....                  | 7         |
| Jelszó:.....                      | 7         |
| <b>Filmek oldal.....</b>          | <b>8</b>  |
| <b>Rólunk oldal.....</b>          | <b>9</b>  |
| <b>Profilom oldal.....</b>        | <b>10</b> |
| Kedvenc filmjeid:.....            | 10        |
| Saját értékeléseid:.....          | 10        |
| <b>Film kártyák.....</b>          | <b>11</b> |
| Előnézet:.....                    | 11        |
| A filmek oldala:.....             | 11        |
| <b>Egyéb funkciók.....</b>        | <b>13</b> |
| Sötét / világos nézet:.....       | 13        |
| Critiqly kabala:.....             | 13        |

# Kezdőlap / Főoldal

Egy rövid betöltő animáció után megjelenik a Critiqly főoldala. Ezen oldal célja bizonyos filmek átlátható rendezése és kategorizálása, illetve a felhasználó megismertetése a weboldal alapvető koncepciójával. Mindezt az alábbi elemek segítik elő:

## Bemutató szöveg:

- Az oldal fő színeiben megjelenő nagyméretű összefoglaló szöveg ami a szövegbuborékokkal kiegészítve ismerteti a projekt lényegét és alapelveit.

## Kiemelt filmek:

- Ez a rész a kínálat első 8 filmjét jeleníti meg.

## Legjobbra értékelt filmek:

- Ebben a szekcióban található a legjobb értékelést elért filmek. Egyszerre 15 filmet jelenít meg, amik között lapozható listával tud a felhasználó navigálni.

## Napi válogatás:

- Ezen a részen a filmek véletlenszerűen vannak kiválasztva, mindentől függetlenül. Célja, hogy a felhasználókhoz ne csak a felkapott és híres filmek kerüljenek el, hanem akár a kevésbé ismertek is.

## Közelgő filmek:

- Ez a lapozható lista (carousel slider) lehetővé teszi a közeljövőben megjelenő filmek egyszerű és gyors áttekintését. A lista neve mellett oldalt elhelyezkedő két gomb lehetővé teszi a film kártyák jobbra vagy balra történő léptetését, amivel a felhasználó 8 hamarosan megjelenő filmet ismerhet meg.

## Navigációt elősegítő gombok:

- A főoldalon 2 helyen is megjelenő gombok (filmek böngészése, összes film) segítenek az oldal böngészése közben gyorsan átugrani a *Filmek* oldalra további tartalmakért.

# Navigációs sáv

A navigációs sáv (navbar) elemei elősegítik a weboldal könnyen és gyorsan történő böngészését bármelyik lapon is van a felhasználó. A Critiqly navigációs sávja az alábbi elemeket tartalmazza:

## Logó:

- A bal felső sarokban található logóra kattintva a felhasználó bármely másik oldalról visszakérül a főoldalra, egy rövid betöltő animáció után.

## Keresősáv:

- A weboldal keresősávja lehetővé teszi a cím vagy kulcsszó alapú film keresést magyar nyelven. A keresés megkezdése az *Enter* gomb lenyomásakor történik. Amennyiben a felhasználó törölni szeretné a beírt szöveget a keresősáv jobb oldalán, az írást elkezdve megjelenő kék X jelzésű gombra kattintva egyszerűen megteheti.

## Főoldal fül:

- A FŐOLDAL jelzésű linkre kattintva a felhasználó könnyen visszajut a kezdőlapra az oldal bármely részéről.

## Filmek fül:

- A FILMEK jelölésű linkre kattintva a felhasználó átjuthat az adatbázisunk összes filmjének kilistázására szolgáló oldalra.

## Rólunk fül:

- A RÓLUNK nevű link eljuttatja a felhasználót egy a projektünk bővebb megismerésére szolgáló weblapra.

## Belépés gomb:

- A gombra kattintva a beléptető oldalra jut a felhasználó.
- Amennyiben a felhasználó be van jelentkezve a BELÉPÉS gomb helyett a egy PROFILOM nevű gomb jelenik meg, mellette a felhasználó nevével. Ez a gomb a felhasználó profiljának oldalára vezet.

## Regisztráció gomb:

- A gombra kattintva a felhasználó a regisztrációs oldalra jut el.

- Amennyiben a felhasználó be van lépve a REGISZTRÁCIÓ gomb helyett egy KIJELENTKEZÉS gomb jelenik meg, ami egyszerűen kijelentkezteti a felhasználót további kattintások nélkül.
-

# Regisztrációs oldal

A weboldal regisztrációs része a megszokott űrlap formát veszi fel. A felhasználó pár adat megadásával itt tud fiókot regisztrálni az oldalunkra. A regisztrációhoz szükséges adatok:

## Név:

- Ebbe a mezőbe egy maximum 255 karakterből álló tetszőleges nevet kell megadnia a felhasználónak. Ez a név jelenik majd meg a regisztráció után a felső sávban, egy rövid üdvözléssel.

## Felhasználónév:

- Ebbe a mezőbe egy szintén 255 karakter hosszú tetszőleges felhasználónevet kell megadni.

## E-mail cím:

- Az E-mail címekre vonatkozó formális elvárások mellett (@ jel) a megadott adatnak egyedinek is kell lennie, azaz még nem regisztrált E-mail címet lehet csak megadni.

## Jelszó:

- A jelszó mezőbe megadott adatnak - ahogy azt a mezőben lévő felirat is jelöli - legalább 8 karakternyi hosszt el kell érnie. Ezen felül a jelszónak legalább 1 nagybetűt és számot is tartalmaznia kell. A mezőben az írás kezdetekor megjelenő szem ikonra kattintva a felhasználó láthatóvá teheti jelszavát, illetve ugyanígy el is fedheti azt.

## Jelszó újra:

- Az ezt megelőző mezőben megadott jelszó pontos másának beírása szükséges a regisztráció utolsó lépéséhez.

Amennyiben bármelyik adat téves, azt a REGISZTRÁCIÓ gomb lenyomása után az űrlapon belül külön üzenet fogja jelezni. Hibátlan adatmegadás esetén a gomb lenyomása után a felhasználó visszakérül a főoldalra, már a regisztrált fiókjában belépve.

Ha a felhasználónak van már fiókja akkor egyszerűen átléphet a belépéshez szükséges oldalra a LÉPJ BE ITT szövegű linkre kattintva.

# Belépés oldal

A belépésre szolgáló weblap a regisztrációs felületet idézi, szintén pár adat megadásával tudnak a már korábban regisztrált felhasználók belépni profiljukba:

## E-mail cím:

- Ide a regisztrációkor megadott E-mail címét kell beírni a felhasználónak.

## Jelszó:

- Ebbe a mezőbe kell megadni a regisztráláskor ellenőrzött és elmentett jelszavát a felhasználónak.

Amennyiben bármelyik adat téves, azt a BELÉPÉS gomb lenyomása után az űrlapon belül külön üzenet fogja jelezni. Hibátlan adatmegadás esetén a gomb lenyomása után a felhasználó visszakerül a főoldalra, már a fiókjában belépve.

Ha a felhasználónak nincs még fiókja akkor egyszerűen átléphet a regisztrációhoz szükséges oldalra a REGISZTRÁLJ ITT szövegű linkre kattintva.

---



## Filmek oldal

Az összes adatbázisunkban lévő film kilistázására szolgáló oldalra a navigációs sávból, illetve a főoldalon elhelyezkedő gomb vagy link lenyomásával is el lehet jutni. Az oldal fő célja a filmek egyszerű átlapozása, amelyek egyesével, úgy nevezett film kártyákon jelennek meg. Amennyiben a felhasználó legörget az oldalon, a lap alján található számozott gombokkal, vagy az egyszerű jobbra vagy balra lapozó gombokkal a következő, egyéb filmeket betöltő oldalra tud ugrani, amíg el nem éri a lista végét.

Ahogy azt az oldal tetején lévő szövegrész is jelöli, a weblapon a filmek műfaja szerinti szűrésre is van lehetőség, a következő főbb kategóriákban:

- Akció
- Vígjáték
- Dráma
- Sci-Fi
- Horror
- Romantikus
- Fantasy

A linkekre kattintva a feliratuk szerinti műfajban szereplő filmek fognak betöltődni az oldalon. Az ÖSSZES jelölésű linkkel a szűrésmentes filmek listázását látja a felhasználó, ami az alapvető állása a filmek oldalnak.

Amennyiben egy film több műfajhoz is tartozik, az összes ráillő műfaj szűrése után meg fog jelenni.

---

## Rólunk oldal

Az oldalt a navigációs sávból lehet elérni, lényege a projekt bemutatása dióhéjban. A látogató megismerheti a weboldalunk lényegét egyszerűen az alábbi témákban a hozzájuk illő gondolat kártyák tanulmányozásával:

- Missziónk
- Közösség
- Jövő
- Adatvédelem
- Exkluzivitás
- Technológia

Ez a személyes hangvételű weblap - eltérően az oldal többi részétől - nem a filmekről vagy az értékelésekről szól, hanem egyéb fontosnak vélt információ átadásáról az oldallal és annak közösségével kapcsolatban.

---

## Profilom oldal

A felhasználó profiljának oldala kizárólag bejelentkezett fiók esetében jelenik meg és a navigációs sávból megközelíthető, az azt jelölő gomb segítségével.

Az oldal betöltésekor egy a felhasználó nevének kezdőbetűjével ellátott profilképpel találkozhatunk, ami alatt megtekinthető a tagság kezdetének időpontja (regisztráció dátuma). Emellett a profil oldal az alábbi 2 felhasználó specifikus adat megjelenítésére alkalmas:

### Kedvenc filmjeid:

- Ez a szekció a felhasználó által kedvencnek jelölt filmeket listázza ki, annak időrendi sorrendjében. Amennyiben a felhasználó még egy filmet sem kedvelt be ez a bal oldali sáv üresen jelenik meg. Bármelyik kedvencre kattintva az adott film oldala jelenik meg.

### Saját értékeléseid:

- A felhasználó, filmeknek adott értékeléseit megjelenítő sáv a kedvencek melletti, jobb oldalon elhelyezkedő szekció. A listázás az értékelés adásának időrendi sorrendjében történik, ahol az értékelt film címe és az adott értékelés (1-től 10-ig csillagok) íródik ki. Ha a felhasználó nem írt még értékelést, az esetben ez a szekció üresen jelenik meg. Bármelyik értékelésre kattintva az adott film oldala jelenik meg.

Mindkét oszlopnál, ha több mint 5 adat szerepel, egy-egy “Összes megjelenítése” gomb jelenik meg, mely lenyomásakor az összes további adat kiírásra kerül.

---

# Film kártyák

## Előnézet:

Az úgynevezett film kártyák a weboldalon a kezdőlap, illetve filmek oldalakon található keretek, amikben 1-1 film jelenik meg a hozzájuk tartozó alábbi adatokkal:

- Poszter
- Értékelések átlaga
- Cím
- Megjelenés éve
- Műfaj(ok)

## A filmek oldala:

Amennyiben a felhasználó rákattint egy film kártyára, betöltődik az ahhoz tartozó film saját oldala. Ezen a weblapon a filmhez tartozó specifikus információkat / adatokat láthatja a látogató. A film kártyákon szereplő adatok mellett a következőkkel találkozhatunk:

### Történet:

- Az angolul ún. plot, egy pár mondatos összefoglaló a film cselekményéről, ami a felhasználó érdeklődésének felkeltésére szolgál.

### Elérhetőség:

- A bejelentkezett felhasználók 5 platform közül szavazhatnak, hogy hol lehet a kiválasztott filmet megtekinteni. A közösség építő jellegét kihasználva tudják ezáltal a felhasználók segíteni egymást. A gombokon látható, hogy hány felhasználó szavazott a platformokra, ezáltal a felhasználó láthatja, hogy melyik streaming platformon találja meg az adott filmet, a legnagyobb eséllyel.
- Minden bejelentkezett felhasználó, filmenként minden platformra csak egyszer tud szavazni. Ha a szavazatát később meggondolja, vagy véletlenül szavazott, akkor a választott platform gombjára újra kattintva a szavazatát vissza tudja vonni.
- Néhány filmnél egy-egy platform ki van emelve, ezek azok a platformok ahol, az adott film biztosan elérhető. Ezt a funkciót nem a felhasználó irányítja, hanem az admin.

## Kedvelés:

- A KEDVENCEKHEZ feliratú gomb teszi lehetővé a felhasználónak, hogy kedvenc filmjeit elmentse, illetve a későbbiekben akár a profil fülben megtekinthesse. A gomb lenyomása után, az oldal újratöltésével már egy más kinézetű ELTÁVOLÍTÁS feliratú gomb jelenik meg az előző helyén, aminek megnyomásával a felhasználó kedvencei közül törlődik az adott film.
- Ez a funkció kizárólag bejelentkezett felhasználóknak elérhető!

## Megosztás:

- A MEGOSZTÁS feliratú gombra kattintva a látogató kimásolhatja az adott film oldalának linkjét. Ezt egy "Link másolva!" tartalmú felugró szövegbuborék erősíti meg.

## Előzetes:

- Azoknál a filmeknél ahol a szinkronos előzetes elérhető, ott a vélemények felett megjelenik a filmhez tartozó előzetes, rákattintva az ajánló elindul.
- Amely filmeknél nem elérhető az előzetes, egy üzenet tájékoztat róla.

## Vélemények:

- Ebben a szekcióban a felhasználó megtekintheti az adott film értékeléseit amit az oldalunkon kapott. Ugyanitt megjelenik a hozzászólások száma, illetve a felhasználóink által adott csillagok átlaga.
- A vélemények kilistázásánál az értékelők nevét láthatjuk az általuk adott csillagok számával, a kommentjükkel (ha van) és a véleményük beküldésének dátumával.
- A saját véleményadásra kizárólag bejelentkezett profillal van lehetőség! Ha nincs bejelentkezve a felhasználó, akkor erre egy gomb figyelmeztet a vélemények felett. Ha a felhasználó bejelentkezett, egyszerűen kiválaszthatja egy 1-től 10-ig terjedő skálán, hogy hány csillagot érdemel szerinte a film, majd akár még szöveges értékelést is adhat rá. A vélemény közzétételéhez a KÜLDÉS gombra kell kattintani.

## Hasonló filmek:

- Minden film oldalának legalján található a Hasonló filmek szekció. Itt egyszerre maximum 4 olyan film kerül megjelenítésre, amelyek műfaja(i) megegyeznek a kiválasztott filmével.

## Egyéb funkciók

### Sötét / világos nézet:

A felhasználói élmény maximalizálásához hozzájárul az oldal sötét / világos nézetének változtatására szolgáló gombunk. Minden oldal jobb alsó sarkában elhelyezkedő, kör alakú gombra nyomva az oldal színei megváltoznak, míg a megszokott elrendezés megmarad. A felhasználó így a kedvére igazíthatja a weboldal megjelenését.

### Film varázsló:

A bal alsó sarokban található a film varázsló melynek szerepét **Cricklee** tölti be. A felhasználó erre a gombra kattintva megjelenítheti a segéd felületet, melynek célja, a film keresési folyamat rövidítése és megkönnyítése. Két kérdés megválaszolása után, három film ajánlatot kap a felhasználó. A kettő kérdésből az első, a film műfajára vonatkozik, míg a második a megjelenési időszakra. A preferált válaszokra való kattintás után, amennyiben vannak ilyen filmek, három véletlenszerű kap a felhasználó a megadott témákban, ha nincs akkor erről egy szöveges kiírás tájékoztat. Az eredményül kapott filmekre kattintva egyenesen az adott film oldalára kerül a felhasználó. Amennyiben nem elégedett az eredménnyel, vagy nem kapott eredmény a felhasználó, a “Varázsolj újra!” gombra kattintva, újratekesheti a keresést.

### Critiqly kabala:

A Critiqly platform egyedi arculatának meghatározó eleme a **Cricklee** néven futó kabalafigura. Integrációja túlmutat az esztétikai szempontokon; célja a felhasználói élmény személyesebbé tétele és a vizuális kommunikáció segítése, az alábbi módokon:

#### Változatos vizuális megjelenés:

- A platform dinamikusan generált logókat használ. Minden oldalbetöltés alkalmával a rendszer véletlenszerűen választ a rögzített variációk közül, így biztosítva a felület folyamatos vizuális frissességét.

#### Intelligens tartalom helyettesítés:

- Olyan esetekben, amikor egy filmhez nem áll rendelkezésre hivatalos előzetes, Cricklee grafikája jelenik meg funkcionális helyőrzőként. Ez a megoldás segít elkerülni az üres felületeket vagy technikai hibaüzeneteket, fenntartva a felület egységes megjelenését.

## Segítségnyújtás:

- A kabala a filmvarázsló központi vizuális eleme. Elsősorban a felhasználói felület megjelenését és hangulatát határozza meg, így fontos szerepet játszik az egységes arculat kialakításában. A funkció vizuális felépítése ehhez a karakterhez igazodik.
-

Fejlesztői



# **Critiqly**

Fejlesztői dokumentáció

# Tartalomjegyzék

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>Tartalomjegyzék.....</b>                                  | <b>2</b>  |
| <b>A projektről röviden.....</b>                             | <b>3</b>  |
| Koncepció:.....                                              | 3         |
| Inspiráció:.....                                             | 3         |
| Projektmunka:.....                                           | 3         |
| A webes felület főbb funkciói:.....                          | 3         |
| Az asztali felület főbb funkciói:.....                       | 4         |
| <b>A projekt alapja.....</b>                                 | <b>7</b>  |
| Adatbázis:.....                                              | 7         |
| Weboldal:.....                                               | 10        |
| Asztali alkalmazás:.....                                     | 12        |
| <b>Technikai ismertető.....</b>                              | <b>15</b> |
| Frontend-ről bővebben:.....                                  | 15        |
| Külső technológia:.....                                      | 16        |
| Speciális UI/UX megoldások és frontend logika:.....          | 17        |
| Cricklee: Rendszerszintű implementáció és eszközkezelés..... | 18        |
| Interaktív Asszisztens (Wizard) architektúra:.....           | 19        |
| Backend-ről bővebben:.....                                   | 20        |
| Requestek:.....                                              | 20        |
| Kontrollerek:.....                                           | 20        |
| Útvonalak (routes):.....                                     | 21        |
| Asztali alkalmazásról bővebben:.....                         | 21        |
| Adatkezelés és binding:.....                                 | 22        |
| Eseménykezelés és életciklus:.....                           | 22        |
| API kommunikáció:.....                                       | 22        |
| Állapotkezelés:.....                                         | 23        |
| UI visszajelzések és működés:.....                           | 23        |
| Architektúrális döntések:.....                               | 23        |
| <b>Tesztelés.....</b>                                        | <b>24</b> |

# A projektről röviden

## Koncepció:

A célunk egy egyszerűen használható, mindenki számára átlátható weboldal megalkotása volt, ahol a felhasználók felfedezhetnek új filmeket vagy akár értékelhetnek régebbieket. Egy olyan felületet szerettünk volna létrehozni, ahol mindenki véleménye számít, amit ízlésesen csillagokkal vagy akár szóban is ki tudnak fejezni a felhasználók.

## Inspiráció:

Oldalunkhoz az inspirációt az IMDb nevű weboldalból merítettük, amely egy filmeket és egyéb médiát tároló internetes adatbázis. Habár vannak hasonló funkciók, igyekeztünk a saját ötleteink megvalósítására terelni a hangsúlyt ezzel is egyedivé téve az oldalunk.

## Projektmunka:

A csoport tagjai:

- Bagonyi Dóra
- Barna Márk
- Baróthi Alex

Csoportmunkához használt felületek:

- Messenger
- Discord

## A webes felület főbb funkciói:

**Dinamikus és reszponzív UI:** Egyedi média-lekérdezésekkel és adaptív rácsrendszerrel biztosítja a hibátlan élményt minden kijelzőméreten, automatikusan átrendezve a navigációt és a médiatartalmakat.



**Aszinkron interakciók:** A szavazási rendszer, a kedvencek funkció és a pontozás az oldal újratöltése nélkül kommunikál a szerverrel, azonnali vizuális visszajelzést adva.

**Állapotvezérelt felület:** A JavaScript réteg dinamikusan módosítja a gombok funkcióját a film állapota szerint.

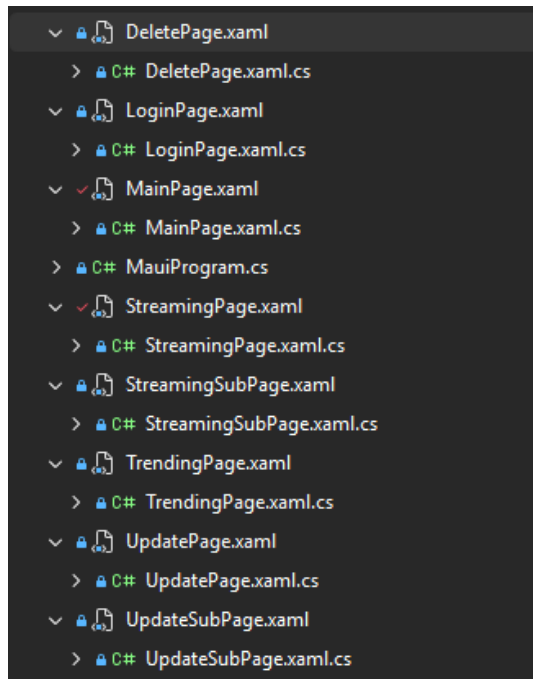
**Kliensoldali adatkezelés:** A localStorage használatának segítségével a böngésző megjegyzi a felhasználó korábbi interakcióit, megelőzve a többszörös szavazást és javítva a sebességerzetet.

**Validáció és biztonság:** Integrált CSRF-védelem és azonnali frontend-szintű jogosultságkezelés, amely értesítésekkel segíti a felhasználót a bejelentkezési folyamatokban.

## Az asztali felület főbb funkciói:

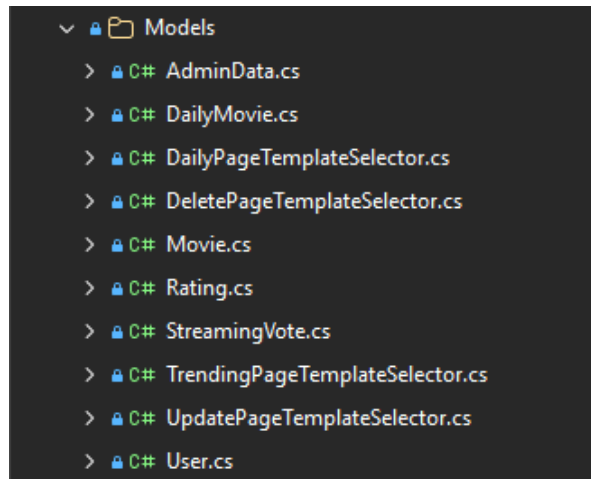
**Alkalmazás szintű állapotkezelés:** Az alkalmazás belső kommunikációja az AppData osztályon keresztül történik, amely statikus adattárolóként működik, és elérhetővé teszi az egész alkalmazásban a kulcsfontosságú adatok (Például filmek, értékelések) elérését a különböző komponensek között.

**Oldal-alapú felépítés:** Az alkalmazás szerkezetében minden funkcionális oldal külön `ContentPage`-ként jelenik meg, melyek saját logikával rendelkeznek a `code-behind` fájlokban. A közöttük lévő navigáció a .NET MAUI Shell mechanizmusán keresztül történik, amely lehetővé teszi az oldalak közötti egyszerű váltást, és a moduláris felépítést.



```
<ShellContent Title="Login" ContentTemplate="{DataTemplate local:LoginPage}" Route="LoginPage"/>
<ShellContent Title="Home" ContentTemplate="{DataTemplate local:MainPage}" Route="MainPage"/>
<ShellContent Title="Daily" ContentTemplate="{DataTemplate local:DailyPage}" Route="DailyPage"/>
<ShellContent Title="Trending" ContentTemplate="{DataTemplate local:TrendingPage}" Route="TrendingPage"/>
<ShellContent Title="Update" ContentTemplate="{DataTemplate local:UpdatePage}" Route="UpdatePage"/>
<ShellContent Title="Delete" ContentTemplate="{DataTemplate local>DeletePage">
<ShellContent Title="UpdateSub" ContentTemplate="{DataTemplate local:UpdateSubPage}" Route="UpdateSubPage"/>
<ShellContent Title="StreamingPage" ContentTemplate="{DataTemplate local:StreamingPage}" Route="StreamingPage"/>
<ShellContent Title="StreamingSub" ContentTemplate="{DataTemplate local:StreamingSubPage}" Route="StreamingSubPage"/>
```

**Modell-alapú struktúra:** Az adatkezelés modell-alapú struktúrára épül, ahol különböző osztályok reprezentálják az egyes entitásokat, például `Movie`, `Rating` vagy `StreamingVote` objektumokat. A modellek nem csak kizárólag adatstruktúráként szolgálnak hanem tartalmaznak állapotjelző mezőket is (Kiválasztottság vagy módosítottság jelzése), melyek a felhasználói felület működését támogatják.



**HTTP-alapú API hívások:** A backenddel való kommunikáció HTTP-alapú API hívásokon át történik, melyeket az alkalmazás bizonyos pontjain HttpClient segítségével valósít meg. Az adatok JSON formában kerülnek feldolgozásra, és a különböző oldalak felelősek a saját frissítési -és adatlekérési műveleteik végrehajtásáért.

**Szabványos konfiguráció és inicializálás:** A konfiguráció a .NET MAUI belépési pontjain történik, különös tekintettel a MauiProgram osztályra, ahol a platformspecifikus viselkedések illetve szolgáltatások kerülnek beállításra, például Windows ablakméretének korlátozása. Az alkalmazás jelenleg a Windows platformra optimalizált, azonban a MAUI architektúra lehetőséget biztosít más platformokra (Android, IOS vagy akár Linux) való kiterjesztését is.

---

# A projekt alapja

## Adatbázis:

A rendszer Laravel keretrendszerre épül, az adatbázis felépítése pedig migrációk segítségével lett kialakítva, így a struktúra átláthatóan kezelhető, és később egyszerűen módosítható vagy továbbfejleszhető.

## TMDB API használata:

Az oldal filmadataihoz a TMDB (The Movie Database) nyilvános filmes API-ját használtuk. Ez egy ismert adatforrás, amely naprakész információkat biztosít filmekről, például címről, műfajról, leírásról, megjelenési dátumról, plakátképről és más egyéb filmekhez kapcsolatos tudnivalókról. Ennek köszönhetően nem kellett manuálisan feltölteni több száz film adatait, hanem az alkalmazás automatikusan külső forrásból tudta betölteni azokat.

**MovieSeeder:** feladata, hogy a TMDB API-ból lekért filmadatokat feltöltse saját adatbázisba. Összesen 25 oldalnyi adat kerül feldolgozásra, így nagyobb mennyiségű film kerülhet az adatbázisba. Minden film esetében ellenőrzi, hogy rendelkezésre állnak-e a szükséges adatok, például az azonosító és a megjelenési dátum. Ezután a műfaj azonosítókából szöveges műfajneveket készít, majd külön API hívással megpróbál hivatalos YouTube előzetest is keresni az adott filmhez. Az updateOrCreate() metódussal elmenti az adatokat az adatbázisba. Ez azt jelenti, hogy ha a film már létezik, akkor frissíti az adatait, ha pedig még nem szerepel benne, akkor új rekordként létrehozza.

## Táblák és felépítésük:

**Users:** A regisztrált felhasználók adatait tárolja

- id: A felhasználó egyedi azonosítója
- name: A felhasználó teljes neve
- username: Felhasználónév
- email: A felhasználó E-mail címe
- email\_verified\_at: Jelzi mikor lett megerősítve az E-mail cím, amennyiben nem lett akkor üres
- password: A felhasználó által választott jelszó
- is\_admin: Azt jelzi, hogy az adott felhasználó admin-e vagy sem
- rememberToken: A bejelentkezés megjegyzéséhez használható
- timestamps: A létrehozás és módosítás idejét tárolja

**Movies:** A filmek adatait tárolja

- id: A filmek azonosítója
- tmdb\_id: Egy egyedi azonosító, amely lehetővé teszi a film összekapcsolását a The Movie Database (TMDB) adatbázissal.
- title: A film címe
- genre: A film műfaja
- plot: A film cselekményének összefoglalása
- releaseDate: A film megjelenésének dátuma
- poster: A film plakátjának az elérési útja
- trailerUrl: A film előzetesének linkje, amennyiben van neki
- streamUrl: A filmhez tartozó, streamingszolgáltatóra mutató link
- timestamps: A létrehozás és módosítás idejét tárolja
- softDeletes: Törlés esetén az adat nemvész el véglegesen, hanem akár visszaállítható

**Ratings:** A filmekre adott értékeléseket tárolja

- id: Az értékelés azonosítója
- movie\_id, user\_id: Összekötés a film és a felhasználó között azonosító alapján
- comment: Opcionális szóbeli értékelés
- stars: Értékelés pontszáma
- timestamps: A létrehozás és módosítás idejét tárolja
- softDeletes: Törlés esetén az adat nemvész el véglegesen, hanem akár visszaállítható

**Daily\_movies:** Tárolja, hogy egy adott napon melyik film volt kiemelve.

- id: Azonosító szám
- movie\_id: A film azonosítója



- **date:** A dátum amelyik napra vonatkozik az ajánlás
- **timestamps:** A létrehozás és módosítás idejét tárolja

```
$table->unique(['movie_id', 'date']);
```

Ez a sor azt biztosítja, hogy ugyanaz a film egy adott napon csak egyszer szerepelhessen a táblában, tehát nem lehet duplikált napi ajánlás ugyanarra a filmre.

**Trending\_movies:** Éppen felkapott filmek listázásához kell

- **id:** Azonosító szám
- **movie\_id:** Hivatkozik a filmre
- **timestamps:** A létrehozás és módosítás idejét tárolja

```
$table->unique(['movie_id']);
```

Ez biztosítja, hogy egy film csak egyszer szerepelhessen

**Admin\_data:** Ez a tábla azt követi, hogy az admin jellegű adatok (napi ajánló, trending lista) mikor lettek utoljára frissítve.

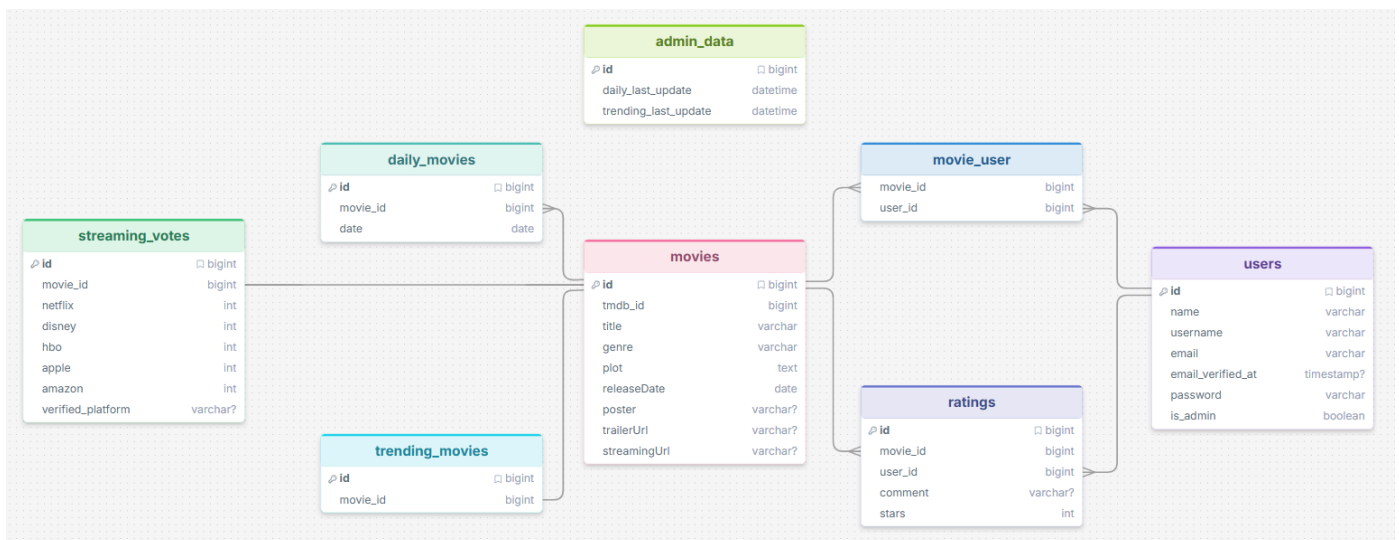
- **id:** Azonosító szám
- **daily\_last\_update:** Mikor lett utoljára frissítve a napi filmek listája
- **trending\_last\_update:** A felkapott filmek listájának frissítése
- **softDeletes:** Törlés esetén az adat nem vész el véglegesen, hanem akár visszaállítható
- **timestamps:** A létrehozás és módosítás idejét tárolja

**Streaming\_votes:** Ez a tábla azt követi, hogy a felhasználók szerint melyik platformon érhető el egy film.

- **id:** Azonosító szám
- **movie\_id:** Hivatkozik a filmre

- netflix: Hány szavazat érkezett a Netflix platformjára
- disney: Hány szavazat érkezett a Disney platformjára
- hbo: Hány szavazat érkezett a HBO platformjára
- apple: Hány szavazat érkezett a Apple platformjára
- amazon: Hány szavazat érkezett az Amazon Prime platformjára
- verified\_platform: jelzi, ha már megerősítették biztosan a platformot
- softDeletes: Törlés esetén az adat nem vész el véglegesen, hanem akár visszaállítható
- timestamps: A létrehozás és módosítás idejét tárolja

### Táblák közötti kapcsolat:



### Weboldal:

A weboldal fejlesztése során a stabilitást, a gyorsaságot és a könnyű karbantarthatóságot tartottuk szem előtt. Ebből kifolyólag a változókat és a fájlokat logikusan neveztük el, valamint a tiszta kód elvét figyelembe vettük. A projekt fejlesztési eszköztárát úgy válogattuk össze, hogy a modern webes elvárásoknak megfelelően, miközben elkerüljük a feleslegesen túlbonyolított keretrendszereket.

### Fejlesztői eszközök:

**Laravel:** A projekt gerincét alkotó keretrendszer, amely a backend logikát, az adatbázis-kezelést és a biztonsági funkciókat látja el.

**Blade templating:** A Laravel saját sablonmotorja, amely lehetővé teszi a tiszta HTML struktúrák és a szerveroldali adatok gyors és biztonságos összekapcsolását.

**JavaScript:** A kliensoldali interakciókért felel. Nem használtunk külső keretrendszert a szavazáshoz és a kedvencekhez, helyette a natív Fetch API segítségével valósítottuk meg az aszinkron kommunikációt.

**CSS3:** A reszponzív elrendezés és a vizuális stílus kialakításához használt modern stíluslap-technológiák.

**Visual Studio Code:** Az elsődleges kódorientált fejlesztői környezet, amely a munkafolyamat nagy részét segítette.

### Főbb frontend komponensek:

A Blade segítségével újrahasznosítható komponenseket hoztunk létre:

**Navbar:** Tartalmazza a keresőt, a navigációs linkeket és a felhasználói állapotot. Mobilon illetve tableten rács alapú elrendezésre vált.

**Movie card:** A főoldalon és a keresési találatoknál megjelenő egységes elem, amely tartalmazza a film poszterét, címét, megjelenési dátumát, a műfajait, illetve a kerekített értékelését.

**Star rating:** Egy interaktív, 10 csillagból álló beviteli mező, amely azonnali vizuális visszajelzést ad a felhasználónak a választott értékről.

**Favorite button:** Aszinkron működésű gomb, amelyet a fetch API segítségével, oldalújrátöltés nélkül adja hozzá vagy távolítja el a filmet a felhasználó listájáról.

**Toast notifications:** Kis felugró ablakok, amelyek a képernyő alján beúszva tájékoztatnak különféle műveletekről vagy eseményekről.

### HTML elemhasználat:

A weboldal kódolása során törekedtünk a szemantikus HTML5 elemek használatára. Ez nemcsak a kód olvashatóságát javítja, hanem segíti a keresőoptimalizálást és a kisegítő technológiák munkáját is.

**nav:** A globális navigáció ezen elembe lett ágyazva, egyértelműen elkülönítve az oldal irányító funkcióit, a tartalmi részekről.

**main:** A page\_shell konténeren belül ez az elem jelöli ki az oldal egyedi, releváns tartalmát, elhatárolva azt a fix fejléc és lábléc elemeitől.

**section:** A logikai összefüggő tartalmi blokkok elkülönítésére szolgál, segítve a dokumentum vázlatának felépítését.

**article:** Az önállóan értelmezhető tartalmi egységeknél, például a filmkártyák esetében alkalmazzuk, biztosítva a komponensek függetlenségét.

**Interaktív elemek:** A <button> tagek kizárólag kliens- vagy szerveroldali funkciókat indítanak, míg az <a> elemeket szigorúan navigációs célokra, belső és külső hivatkozások kezelésére korlátoztuk.

**Űrlapkezelés:** Az adatok beküldésére szolgáló interfészeknél szabványos űrlap-elemeket használtunk. A Laravel backenddel való konzisztenciát a pontos name attribútumok és a type="submit" típusú vezérlők biztosítják.

**Vizuális elemek:** A médiatartalmak megjelenítése során minden esetben definiáltuk az alt attribútumokat a hozzáférhetőségi irányelveknek megfelelően.

**Strukturális elemek:** Az általános tárolókat kizárólag elrendezési célokra alkalmazzuk, ahol az elemnek nincs önálló jelentéstartalma.

## Asztali alkalmazás:

Az asztali alkalmazás fejlesztése során a stabil működést, az átlátható felépítést és a könnyű továbbfejlesztheséget tartottuk szem előtt. Ennek megfelelően az alkalmazás szerkezetét logikusan elkülönülő oldalakra bontottuk, valamint egységes elnevezési konvenciókat alkalmazzuk a változók, modellek és komponensek esetében. A kód kialakításakor törekedtünk a tiszta kód elveinek betartására, hogy a projekt hosszabb távon is jól karbantartható maradjon.

## Fejlesztői eszközök:

**Visual Studio:** Az elsődleges fejlesztési környezetként a Visual Studio 2026 került használatra, amely teljes körű támogatást biztosít a .NET alapú alkalmazások fejlesztéséhez, hibakereséshez és futtatásához. Az IDE beépített eszközei megkönnyítik a többplatformos alkalmazások kezelését és tesztelését.

**C#:** A fejlesztés során használt programozási nyelv a c# amely az alkalmazás üzleti logikájának megvalósításáért felel. A .NET ökoszisztémával való szoros integrációja lehetővé teszi a stabil, jól strukturálható és könnyen karbantartható kód írását.

**XAML:** A felhasználói felület deklaratív leírására a XAML szolgál, amely segítségével a vizuális elemek és azok elrendezése külön kezelhető a logikai működéstől. Ez a megközelítés elősegíti az átláthatóbb kódszerkezetet és a felület könnyebb módosíthatóságát.

**.NET MAUI keretrendszer:** Az alkalmazás alapját a .NET MAUI keretrendszer biztosítja, amely lehetővé teszi a natív felhasználói felületek létrehozását egyetlen közös kódbázisból, amely különösen előnyös a jövőbeli bővíthetőségek szempontjából.

**Git, GitHub:** A verziókezeléshez a Git és Github rendszer került használatra, amely lehetővé teszi a folyamatok, módosítások nyomonkövetését és a biztonságos együttműködést.

## Főbb komponensek:

**MainPage:** Az alkalmazás központi felülete, ahol a filmek, értékelések és admin adatok betöltése történik. Innen érhetők el a további funkcionális oldalak.

**DailyPage:** A napi kiemelt filmek kiválasztására szolgáló felület, ahol a felhasználó listából jelölhet ki elemeket, majd mentheti a változtatásokat.

**TrendingPage:** A trending filmek kezelésére szolgáló oldal, amely hasonló kiválasztási logikát alkalmaz, mint a DailyPage, de külön adathalmazzal dolgozik.

**UpdatePage / UpdateSubPage:** A filmek szerkesztésére szolgáló komponensek, ahol a felhasználó kiválaszthat egy elemet, majd részletesen módosíthatja annak adatait.

**DeletePage:** A filmek törlésének kezelésére szolgáló felület, amely lehetővé teszi több elem kijelölését és egyidejű eltávolítását.

**StreamingPage / StreamingSubPage:** A streaming szavazatok kezelésére és ellenőrzésére szolgáló komponensek, ahol a felhasználó validálhatja a beérkezett adatokat és frissítheti a kapcsolódó információkat.

**Movie listaelemek:** Egységes megjelenítésű listaelemek, amelyek egy film alapadatait jelenítik meg, valamint különböző állapotokat (kiválasztott, módosított, törölt) vizuálisan is jelzik.

**Műveleti gombok:** Interaktív elemek, amelyek különböző admin műveleteket indítanak (pl. mentés, frissítés), és vizuális visszajelzést adnak az aktuális állapotról.

**StatusLabel:** Szöveges visszajelző komponens, amely információt nyújt a legutóbbi műveletekről vagy az adatok frissítési állapotáról.

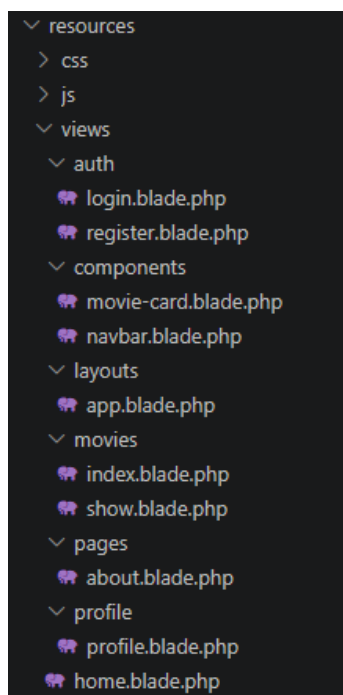
---

# Technikai ismertető

## Frontend-ről bővebben:

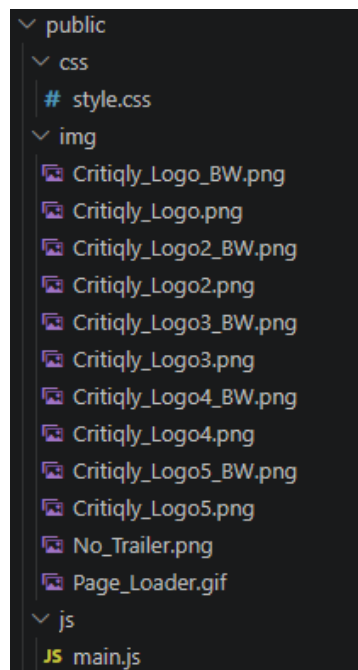
A frontend felépítése moduláris, elkülönítve a statikus stílusokat, a dinamikus szkripteket és a szerveroldali nézeteket.

- **public/resources/views:** Itt találhatóak a Blade fájlok, amelyek az oldal szerkezetét adják.
- **layouts/ app.blade:** Ez tartalmazza az oldal alap elrendezését.
- **auth/ login.blade:** Ez a bejelentkezés felülete.
- **auth/ register.blade:** Ez a regisztrációs felület, a fiók nélküli felhasználóknak.
- **components/ movie-card.blade:** Az oldalon megjelenő film kártyák sablonja.
- **components/ navbar.blade:** A navigációs sáv váza.
- **pages/ about.blade:** A Rólunk oldal kódja.
- **movies/ index.blade:** Az összes filmet megjelenítő aloldal.
- **movies/ show.blade:** A különálló egyéni oldalai a filmeknek.
- **profile/ profile.blade:** A profil aloldal.
- **home.blade:** Ez a főoldal, más néven landing page, fájlja.



Ugyanúgy a public mappában, azon belül a css és js mappákban található a style.css fájl, ez szabja meg a teljes oldal kinézetét, illetve a script.js fájl ami meg az egyéb működésekért felel.

Az img mappa tartalmazza az oldalon megjelenő képeket, például a töltőképet vagy a váltakozó logókat.



## Külső technológia:

### Swiper.js:

**Használat oka:** A reszponzív, érintésérzékeny görgetés saját fejlesztése időigényes és kockázatos lenne. A Swiper.js stabil, böngészőfüggetlen és natív alkalmazás-szerű élményt nyújt.

**Felhasználása:** A kezdőoldali filmkategoróriák és a részletes adatlapokon található „Hasonló filmek” szekciók horizontális listázására használtuk.

**Jellege:** Egy könnyűsúlyú, külső JavaScript könyvtárat integráltunk, amely nem igényel más függőséget, így gyors marad az oldalbetöltés.

### Működtetése:

1. A CDN-en keresztül behúzott könyvtárat a Blade sablonokban definiált HTML osztályokkal kötöttük össze.



2. A JavaScript konfigurációban töréspontokat határoztunk meg, így a felület automatikusan alkalmazkodik: mobilon kevesebb, asztali nézetben több filmkártyát jelenít meg egyszerre.

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/swiper@11/swiper-bundle.min.css" />
```

## Speciális UI/UX megoldások és frontend logika:

### Preloader:

A weboldal kódjában helyet kapott egy egyedi betöltőfelület, amely a teljes oldalstruktúra és az erőforrások (pl. Swiper.js, stíluslapok) betöltéséig látható.

- **Működése:** A HTML váz tetején elhelyezett preloader réteg takarja az oldalt, amíg a JavaScript window.load eseménye le nem fut.
- **Célja:** Megakadályozza a „Flash of Unstyled Content” jelenséget, vagyis azt, hogy a felhasználó másodpercekig egy szétesett, stílusok nélküli oldalt lásson.

### Témakezelés:

A felhasználói kényelem érdekében beépítésre került egy téma-váltó funkció, amely a jobb alsó sarokban található theme\_toggle gombbal vezérelhető.

- **Technikai megvalósítás:** A gomb megnyomásakor a JavaScript egy globális CSS osztályt kapcsolgat a body elemen. A stíluslap CSS változókat használ, így a színek (háttér, szöveg, kártyák) azonnal, villanásmentesen változnak meg.
- **Perzisztencia:** A rendszer a localStorage technológiát használja, így a böngésző megjegyzi a választott témát, és az oldal újratöltése vagy későbbi látogatás esetén is az marad érvényben.

### Központi értesítési rendszer:

A felület alján elhelyezett notification konténer felelős az aszinkron visszajelzések megjelenítéséért.

- **Funkció:** Amikor a felhasználó interakcióba lép az oldallal, a háttérben futó AJAX kérés választ kap a szervertől, amit a rendszer egy elegáns felugró üzenetben jelenít meg.

- **UX előny:** Nincs szükség zavaró felugró ablakokra, az értesítés pár másodpercig látható, majd automatikusan, animációval tűnik el, nem szakítja meg a böngészést.

### CSRF biztonsági integráció:

Bár a CSRF védelem a Laravel beépített funkciója, a frontend oldalon külön konfigurációt igényelt.

**Megvalósítás:** A `<meta name="csrf-token">` tag segítségével a JavaScript minden egyes aszinkron kérés fejlécébe automatikusan beilleszti a biztonsági kulcsot. Ez garantálja, hogy csak a mi oldalunkról érkező kéréseket fogadja el a szerver, megvédve a felhasználókat a rosszindulatú külső támadásoktól.

```
<meta name="csrf-token" content="{{ csrf_token() }}">
```

### Reszponzív navigáció:

A frontend kiemelt figyelmet fordít a mobilfelhasználókra. A `components.navbar` és a média-lekérdezések segítségével a felület adaptív.

- **Mobil nézet:** A navigációs menü és a gombok elrendezése optimalizált, pl. nagyobb érintési felületek, átrendezett keresőmező.
- **Asztali nézet:** Kihasználja a nagyobb szélességet, így több filmkártya és részletesebb információ jelenik meg egyetlen képernyőn.

```
@media (max-width: 1024px)
```

```
@media (max-width: 768px) @media (max-width: 480px)
```

## Cricklee: Rendszerszintű implementáció és eszközkezelés

A kabalafigura integrációja szoftveres szinten két pillérre épül: egy kliensoldali véletlenszerűsítő algoritmusra és egy szerveroldali logikai feltételrendszerre.

### Kliensoldali dinamikus logókezelés:

A rendszer a márkaelemek betöltésénél nem statikus elérési utat használ, hanem egy JavaScript alapú szelekciós logikát:

- **Algoritmus:** Egy `Math.random()` alapú generátor határozza meg a betöltendő vizuális variánst.
- **Állapotkezelés:** A script szinkronban működik a sötét/világos mód beállításával. Az `updateLogo()` függvény automatikusan a kiválasztott variáns színvilágban illeszkedő verzióját rendeli az `img` elem `src` attribútumához.

### Szerveroldali fallback-mechanizmus:

A kabala funkcionális szerepet kap a hiányzó adatállományok kezelésében a Blade sablonmotoron keresztül:

- **Logikai kapu:** A rendszer a `movies.show` nézetben ellenőrzi a trailer URL meglétét.
- **Kivételkezelés:** Amennyiben a lekérdezett rekord `trailer_url` mezője null értéket ad vissza, a backend automatikusan a dedikált kabala grafikát rendereli a videólejátszó konténerébe. Ez a fejlesztői döntés javítja a hibatűrést és csökkenti a hiányos adatlapokból adódó negatív UX-hatást.

### Interaktív Asszisztens (Wizard) architektúra:

Az asszisztens modul egy lebegő (floating) UI komponens, amely közvetlen kapcsolatot teremt a felhasználói igények és az adatbázis-lekérdezések között:

- **Dinamikus UI:** A kliensoldali logika `data-step` attribútumok segítségével kezeli a folyamat állapotát (Wizard steps), lehetővé téve a többoldalas interakciót egyetlen modal ablakon belül.
- **Összetett lekérdezési logika:** A backend képes kezelni a többértékű műfaji szűrőket. Az `explode()` függvénnyel darabolt bemeneti stringek alapján a rendszer `orWhere` és `LIKE` operátorokkal végzi el a komplex keresést, növelve a találati relevanciát.
- **Vizuális integráció:** Az asszisztens két különálló vizuális állapotot használ: egy kompakt "Pill" trigger az inicializáláshoz és egy bővített kártyanézetet az interakcióhoz.

## Backend-ről bővebben:

### Requestek:

**StoreRatingRequest** osztály a filmértékelések beküldés előtti ellenőrzésére szolgál. Feladata, hogy csak megfelelő és érvényes adatokat engedjen tovább a vezérlő felé. Az `authorize()` metódus ellenőrzi, hogy a felhasználó be van-e jelentkezve, tehát csak hitelesített felhasználók küldhetnek be értékelést. A `rules()` metódus tartalmazza az adatellenőrzési szabályokat: az értékelés megadása kötelező, 1 és 10 közötti értéket kell tartalmaznia, a szöveges vélemény pedig legfeljebb 1000 karakter hosszú lehet.

```
public function rules(): array
{
    return [
        "rating" => "required|min:1|max:10",
        "content" => "max:1000"
    ];
}
```

### Kontrollerek:

**AuthController** a felhasználói hitelesítésért felelős vezérlő. Kezeli a regisztrációs és bejelentkezési oldalak megjelenítését, valamint az új felhasználók létrehozását és a belépési adatok ellenőrzését. Regisztráció során adatellenőrzést végez, például ellenőrzi az e-mail cím egyediségét és a jelszó erősségét. Sikeres bejelentkezés után munkamenetet indít, kijelentkezéskor pedig biztonságosan lezárja azt.

```
$validated = $request->validate([
    'name' => 'required|string|max:255',
    'username' => 'required|string|max:255',
    'email' => 'required|email|unique:users',
    'password' => [
        'required',
        'min:8',
        'regex:/^(?=.*\d)(?=.*[Ll])(?=.*[Lu]).{8,}$/u',
        'confirmed'
    ]
]);
```

**MovieController** a filmekkel kapcsolatos főbb funkciókat kezeli. Feladata a filmek listázása, keresése és műfaj szerinti szűrése, valamint az egyes filmek részletes adatlapjának megjelenítése. Kezeli a főoldal tartalmát is, ahol kiemelt, napi, trendi és hamarosan megjelenő filmek jelennek meg. Emellett lehetőséget biztosít kedvencek kezelésére, streaming platformokra történő szavazásra, valamint filmajánló funkcióra is.

```

public function home()
{
    $featured = Movie::latest()->withAvg('ratings', 'stars')->take(8)->get();
    $trendingMovies = TrendingMovie::with(['movie.ratings'])->get();
    $dailyMovies = DailyMovie::with(['movie.ratings'])->whereDate('date', today())->get();
    $upcomingMovies = Movie::where('releaseDate', '>', now())->withAvg('ratings', 'stars')->orderBy('releaseDate', 'asc')->take(8)->get();

    return view('home', compact('featured', 'trendingMovies', 'dailyMovies', 'upcomingMovies'));
}

```

**RatingController:** a filmek értékeléseinek kezeléséért felel. A store metódus segítségével a bejelentkezett felhasználók csillagos értékelést és szöveges véleményt küldhetnek be egy adott filmhez. Az adatokat ellenőrzés után elmenti az adatbázisba. Ez biztosítja a filmek közösségi értékelési rendszerének működését.

```

$rating = Rating::create([
    "stars" => $validated['rating'],
    "comment" => $validated['content'],
    "user_id" => auth()->id(),
    "movie_id" => $id
]);

```

## Útvonalak (routes):

**web.php:** a fájl a webes felülethez tartozó útvonalakat kezeli, tehát azokat az oldalakhoz kapcsolódó route-okat, amelyeket a felhasználók böngészőből érnek el. Itt található a regisztráció, bejelentkezés, kijelentkezés kezelése, valamint a főoldal, filmek listája és az egyes filmek adatlapjai. A bejelentkezett felhasználók ezen keresztül tudnak értékelést leadni, kedvencnek jelölni filmeket vagy szavazni streaming platformokra. Tartalmaz továbbá statikus oldalakat is, például a „Rólunk” oldalt, illetve a felhasználói profil oldal útvonalát.

```

Route::get('/rolunk', function () {
    return view('pages.about');
})->name('about');

```

## Asztali alkalmazásról bővebben:

Az asztali alkalmazás felépítése moduláris, elkülönítve a felhasználói felületet (XAML), az alkalmazáslogikát (C# code-behind) és az adatmodelleket.

### Pages (nézetek)

Az alkalmazás főbb nézetei külön .xaml fájlokban kerülnek definiálásra:

- MainPage.xaml: A főoldal, ahol az adatok inicializálása történik
- DailyPage.xaml: A napi filmek kiválasztására szolgáló felület
- TrendingPage.xaml: A trending lista kezelése
- UpdatePage.xaml: Filmek kiválasztása szerkesztéshez
- UpdateSubPage.xaml: A kiválasztott film részletes módosítása
- DeletePage.xaml: Filmek törlésének kezelése
- StreamingPage.xaml: Streaming szavazatok áttekintése
- StreamingSubPage.xaml: Streaming adatok módosítása

Minden nézethez tartozik egy .xaml.cs fájl, amely az adott oldal működéséért és eseménykezeléséért felel.

## Adatkezelés és binding:

Az alkalmazás adatkezelése modell-alapú struktúrára épül, ahol az egyes entitásokat külön osztályok reprezentálják (Movie, Rating, StreamingVote, stb.).

**Binding működése:** A felhasználói felület és az adatok közötti kapcsolat binding mechanizmusokon keresztül valósul meg. A modellekben található állapotmezők (pl. kiválasztottság, módosítottság) automatikusan befolyásolják a megjelenést.

**Lista alapú megjelenítés:** A filmek listák formájában jelennek meg, ahol ugyanaz az adatstruktúra több különböző nézetben is felhasználható eltérő logikai műveletekkel.

## Eseménykezelés és életciklus:

Az alkalmazás működése eseményvezérelt.

**Oldal betöltés:** Az OnAppearing metódus felel az adatok betöltéséért és a felület frissítéséért minden navigáció során.

**Felhasználói interakciók:** A gombok és egyéb vezérlők eseménykezelőkön keresztül indítanak műveleteket (pl. mentés, kiválasztás, frissítés), amelyek közvetlenül módosítják az alkalmazás állapotát.

## API kommunikáció:

Az alkalmazás a backend rendszerrel HTTP-alapú API-n keresztül kommunikál.

**Megvalósítás:** Az API-hívások HttpClient segítségével történnek, aszinkron módon.

**Adatfeldolgozás:** A válaszok JSON formátumban érkeznek, majd deszerializálás után közvetlenül beépülnek az alkalmazás állapotába.

**Működés:** Az egyes oldalak saját hatáskörben végzik az adatlekérést és frissítést, nincs külön központi service réteg.

## Állapotkezelés:

Az alkalmazás globális állapotát az AppData osztály kezeli.

**Szerepe:** Egységes hozzáférést biztosít az adatokhoz (filmek, értékelések, kiválasztott elemek, admin adatok) az alkalmazás teljes területén.

**Működés:** Az oldalak közvetlenül olvasnak és írnak ebbe az állapotba, amely azonnal hatással van a megjelenítésre.

## UI visszajelzések és működés:

**Vizuális visszacsatolás:** A felhasználói műveletek során az alkalmazás különböző vizuális változásokat alkalmaz (pl. gombszínek, kijelölések), amelyek jelzik az aktuális állapotot.

**Állapotjelzés:** A StatusLabel komponens szöveges visszajelzést ad a felhasználónak a műveletek eredményéről vagy az adatok frissítéséről.

**Többes műveletek:** Az alkalmazás támogatja több elem egyidejű kezelését (pl. kijelölés és tömeges módosítás vagy törlés).

## Architektúrális döntések:

Az alkalmazás nem alkalmaz teljes MVVM architektúrát vagy külső UI könyvtárakat.

**Indoklás:** A projekt méretéhez igazodva egy egyszerűbb, közvetlenebb megközelítés került kiválasztásra, amely gyorsabb fejlesztést és könnyebb átláthatóságot biztosít.

**Előny:** Kevesebb absztrakció, gyorsabb hibakeresés és egyszerűbb karbantarthatóság.

# Tesztelés

A Laravel keretrendszer beépített tesztelési lehetőségeit használtuk a webalkalmazásunk ellenőrzésére, mivel azok támogatják a különböző funkciók automatizált vizsgálatát. A projekt során a teszteléshez a Laravel által biztosított Feature teszteket használtunk, amelyek felhasználói működés szempontjából vizsgálják az alkalmazást. Az alábbi tesztek készültek:

- **Regisztráció működése (test\_user\_can\_register)**  
Készült teszt arra, hogy egy új felhasználó sikeresen tud-e regisztrálni az oldalon. A teszt ellenőrzi, hogy a megfelelő adatok megadása után létrejön-e a felhasználó az adatbázisban.
- **Bejelentkezés (test\_user\_can\_login)**  
A bejelentkezési folyamat is tesztelve lett. Ennek során a rendszer azt vizsgálja, hogy egy korábban létrehozott felhasználó a helyes adatok megadásával be tud-e jelentkezni.
- **Film lista betöltése (test\_movies\_page\_loads)**  
Teszt készült a filmek listázását végző oldalra is. Ez azt ellenőrzi, hogy az oldal megfelelően betöltődik, és a felhasználó eléri a filmek megjelenítését.
- **Keresés működése (test\_movie\_search\_returns\_results)**  
A keresési funkció tesztje azt vizsgálja, hogy egy adott címre történő keresés esetén a rendszer visszaadja-e a megfelelő találatokat. Ez fontos része az oldal használhatóságának.
- **Szűrés műfaj szerint (test\_movies\_can\_be\_filtered)**  
A műfaj szerinti szűrés szintén ellenőrzésre került. A teszt során a rendszernek csak az adott kategóriába tartozó filmeket kell megjelenítenie.
- **Film értékelése (test\_user\_can\_rate\_a\_movie)**  
Készült teszt arra is, hogy a bejelentkezett felhasználók tudnak-e értékelést leadni egy filmre. A teszt ellenőrzi, hogy az értékelés megfelelően elmentésre kerül-e az adatbázisban.

```
PASS Tests\Feature\AuthTest
✓ user can register
✓ user can login

PASS Tests\Feature\MovieTest
✓ movies page loads
✓ movie search returns results
✓ movies can be filtered

PASS Tests\Feature\RatingTest
✓ user can rate a movie
```